BATLOUNGE / SOFTWARE PROJECT GOVERNANCE

Contribution Guide

DEVELOPMENT WORKFLOW • CODE CONVENTIONS • PULL REQUEST QUALITY

A practical, repeatable standard for making focused, secure, maintainable changes to the Batlounge Fruits Distribution Management System.

Project context

This guide supports collaborative development across the React frontend, Express/Node.js backend and MySQL data layer. It is intended to help contributors work consistently, protect business rules and submit changes that are easier to review.

DIWA2110 / WEB APPLICATION DEVELOPMENT

Use this document as a team reference. Adapt commands and branch naming to match the repository’s actual scripts and conventions.

How to use this guide

This document expands the contribution outline into a practical workflow. It describes the expected habits before coding, while implementing a change, and before asking for review.

The guide is organized into these sections

- 1. Contribution principles and scope
- 2. Development workflow from branch to review
- 3. Code conventions for frontend, backend and database work
- 4. Security, authorization and business-rule consistency
- 5. Stock and order transaction guidance
- 6. Documentation and repository hygiene
- 7. Local verification commands and review evidence
- 8. Pull request checklist
- Appendix A. Example branch and commit naming
- Appendix B. Pull request description template
- Appendix C. Common review findings

> Core standard
> A contribution is ready for review when it is focused, follows the project's existing patterns, protects the Admin/Sales access matrix, validates data on both sides of the API, documents meaningful behavior changes, and passes the available checks.

Repository-specific note

The exact folder names and npm scripts may differ in your repository. The examples below assume separate frontend and backend directories, a frontend build script, and a backend JavaScript entry point. Check each package.json and project README before running commands.

1. Contribution principles

Good contributions improve the system without making it harder to understand, test or maintain.

Focused scope

Each branch and pull request should solve one feature, defect or clearly related group of changes. Avoid unrelated refactoring in the same change.

Understand before changing

Read the relevant component, route, controller, database schema and API documentation before implementing a modification.

Consistency across layers

The frontend and backend should agree on field names, required values, validation rules, status values and error behavior.

Security by design

Authentication identifies the user; authorization decides what that user is allowed to do. The backend must enforce permissions for protected operations.

Data integrity

Use parameterized SQL, appropriate constraints and transactions for related stock/order updates.

Reviewability

Prefer clear names, small functions and limited diffs. A reviewer should be able to understand why the change exists and how it was tested.

> A useful test
> If a reviewer cannot quickly answer “what changed, why did it change, and how was it tested?”, improve the branch, code structure or pull request description before requesting review.

2. Development workflow

Use the same repeatable sequence for features and fixes. The sequence lowers integration risk and makes review outcomes more predictable.

Step 1 — Create a focused branch

Start from the team’s current integration branch, pull the latest changes, and create a descriptive branch for one task. Avoid committing directly to a shared protected branch.

Step 2 — Read the relevant documentation

Review the README, API route/controller, frontend screen, database schema and any related issue or assignment requirement. Identify the current behavior before changing it.

Step 3 — Define the change

Write down the expected behavior, affected screens/endpoints/tables, permissions involved, validation rules and acceptance criteria.

Step 4 — Implement the smallest coherent change

Follow existing components and service patterns. Keep controllers, routes and UI components focused. Avoid unrelated formatting or broad rewrites.

Step 5 — Keep layers aligned

Update frontend validation, backend validation, API responses and database constraints where needed. Make error messages understandable and consistent.

Step 6 — Test normal and failure paths

Test valid input, invalid input, loading/error states, role restrictions, empty results and relevant stock/order edge cases.

Step 7 — Update documentation

Document new environment variables, API changes, schema changes, behavior changes or setup instructions where relevant.

Step 8 — Run local checks

Build the frontend, run the backend syntax check and any relevant tests. Resolve failures caused by the change before requesting review.

Step 9 — Review your own diff

Inspect changed files, generated output, secrets, temporary data and unrelated formatting. Confirm the branch contains only intended changes.

Step 10 — Open a pull request

Describe the problem, implementation, testing evidence, database/API impact and any known limitations. Respond to review feedback and rerun checks after fixes.

3. Branches and commits

Branch and commit names should make the purpose of a change clear in history and code review.

Suggested branch naming

Type	Example	Use it for
Feature	feature/vendor-search	New user-facing capability
Bug fix	fix/order-total-validation	Correcting incorrect behavior
Documentation	docs/api-setup	Guides, README or API docs
Refactor	refactor/order-controller	Internal structure without intended behavior change

Commit message guidance

- Use an action-oriented summary, such as Validate order item quantities.
- Keep each commit logically coherent where practical.
- Avoid vague messages such as updates, stuff or final changes.
- Do not include secrets, passwords, tokens or database dumps in commits.

> Before branching
> Confirm the repository’s required base branch and team workflow. The names above are recommendations; do not create a parallel naming scheme if the project already has one.

4. Code conventions

Readable code reduces defects and shortens the time needed for another developer to understand and safely modify the system.

Frontend: React, Tailwind CSS and Lucide

- Use descriptive component, state and handler names such as VendorForm, isSubmitting and handleCreateOrder.
- Keep components focused on one screen section or responsibility; move reusable UI into shared components when it improves consistency.
- Use the existing Tailwind component patterns, spacing scale and responsive breakpoints rather than introducing a separate styling approach without reason.
- Use Lucide icons consistently. Prefer icons that communicate a clear action or status; do not rely on an icon alone when a text label is needed for clarity.
- Handle loading, empty, success and error states. Disable repeated submission when appropriate and provide useful validation feedback.
- Do not treat frontend checks as a security boundary; users can bypass browser-side code.

Backend: Express and Node.js

- Keep routes responsible for mapping endpoints and middleware, controllers focused on request/response flow, and business logic in an appropriate service/helper layer when complexity warrants it.
- Use clear HTTP status codes and predictable response shapes consistent with the existing API.
- Validate request body, parameters and query values before using them.
- Centralize authentication and role-check middleware rather than copying inconsistent permission checks across routes.
- Avoid returning stack traces, SQL details, secrets or internal implementation details to clients.
- Use async error handling consistent with the project’s Express version and established middleware.

Database and SQL

- Use parameterized queries for values supplied by a request; never concatenate user-controlled values into SQL.
- Use clear table and column names consistent with the current schema.
- Use foreign keys and appropriate unique/check constraints where supported and aligned with the design.
- Avoid silently changing existing data assumptions. Document schema changes and provide a migration or repeatable setup update if the project uses them.
- Select only the fields required by a response when practical, and use transactions for related multi-step changes.

5. Validation, authentication and permissions

Validation and authorization must remain consistent across the UI, API and database.

Validation contract

Layer	Responsibility	Example
Frontend	Immediate feedback and usability	Required fields, positive quantity, clear inline error
Backend	Trusted enforcement for every request	Reject missing references, invalid statuses and insufficient stock
Database	Structural integrity for stored records	Foreign keys, unique constraints and appropriate data types

Admin and Sales access matrix

Capability	Admin	Sales	Review question
Master data management	Allowed	Restricted as defined	Are create/edit/delete endpoints protected?
Stock and purchases	Allowed	As defined by project	Can Sales call a restricted API directly?
Vendors and orders	Allowed	Operational access	Are allowed actions limited correctly?
Reports	Allowed	As defined by project	Does the backend enforce report permissions?

> Authorization rule
> The sidebar is a user-interface convenience, not a security boundary. Protected backend endpoints must verify authentication and enforce the appropriate role or permission for the requested action.

JWT handling reminders

- Never commit JWT secrets or other credentials.
- Handle missing, expired or invalid tokens consistently.
- Do not trust a role value supplied freely by the client; derive authorization from trusted server-side identity/session data.
- On logout, clear client-held authentication state according to the application’s design.

6. Stock and order transaction safety

Stock and order changes affect each other. A partial update can leave the system in an inconsistent state, so these paths deserve extra review.

When to use a database transaction

- Creating an order with several order-item records when the records must be saved together.
- Completing an order and deducting stock for its line items.
- Recording a purchase and updating the corresponding product stock.
- Changing order status where the status update and stock movement must remain consistent.

Recommended transaction sequence for completion

1. Begin a database transaction.
2. Load the order and verify it exists and is in a status that permits completion.
3. Check the ordered products and quantities against available stock using a concurrency-safe approach appropriate to the schema.
4. Apply stock deductions for all order items.
5. Update the order status to completed.
6. Commit only if every required operation succeeds; otherwise roll back and return a clear error.

> Avoid double deduction
> A repeated completion request should not deduct stock again. Validate allowed status transitions and make the operation safe against duplicate submissions or retries.

Cancellation behavior

Agree on the project's stock policy and implement it consistently. If stock is deducted only when an order is completed, cancelling an uncompleted order should not deduct stock. If stock is reserved or deducted earlier, cancellation may need to release or restore it. Document the implemented rule so frontend behavior, backend logic and tests agree.

Test the important edge cases

- Quantity is zero, negative, missing or not numeric.
- Requested quantity exceeds available stock.
- Two requests attempt to consume the same limited stock concurrently.
- An order is completed twice or a stale status is submitted.
- A database failure occurs partway through the workflow.
- An order is cancelled before and after the point where stock changes.

7. Documentation and repository hygiene

Keep the repository safe to share and easy for another developer to set up.

Update documentation when behavior changes

- Document new or changed API endpoints, request fields, response shapes, status codes and permissions.
- Document database schema changes, migrations, seed data or setup steps.
- Update the README when installation, environment configuration, scripts or normal workflows change.
- Explain new business rules that affect stock, orders, cancellation, reporting or roles.
- Remove outdated instructions when they no longer match the implementation.

Do not commit

- .env files containing local configuration or secrets.
- Credentials, tokens or private keys in source, logs, screenshots or documentation.
- node_modules/ or generated dependency directories.
- Build output unless the project explicitly requires versioning it.
- mysql-data/ runtime files or local database storage artifacts.
- Temporary exports, debug logs, local dumps or unrelated generated files.

> If a secret is committed accidentally
> Removing it in a later commit is not enough because it may remain in repository history. Notify the project maintainer, rotate or revoke the exposed secret, and follow the repository's cleanup procedure.

Use .gitignore thoughtfully

Confirm the repository ignores local environment files, dependency folders, build output and database runtime directories. Do not add broad ignore patterns that accidentally hide important source files, migrations or lockfiles.

8. Local verification and review evidence

Run the project’s checks from the appropriate directories. If a command differs in package.json, use the repository’s documented equivalent.

Frontend build



A successful build confirms the frontend compiles and bundles according to the configured build process. It does not replace manual checks of responsive layouts, API behavior or authorization.

Backend JavaScript syntax check



The --check option parses JavaScript without running the server. Use the correct entry point or check the changed JavaScript files as appropriate. Syntax checking does not test database connectivity, API correctness or business rules.

Suggested verification matrix

Area	Minimum review evidence
Functional	The requested feature works on a normal valid path.
Validation	Missing, invalid and boundary values are handled clearly.
Permissions	Admin and Sales can perform only the actions defined for their roles.
Data integrity	Related records remain consistent after success and failure.
UI quality	Loading, error and empty states are handled; responsive layout is checked.
Build/syntax	Frontend build and backend syntax checks pass.
Documentation	API, schema and setup notes are updated where required.

9. Pull request checklist

Complete this checklist before requesting review. If an item does not apply, explain why in the pull request rather than silently skipping it.

☐ Requirements — The feature matches the assignment or issue requirement and acceptance criteria.

☐ Scope — The change is focused; unrelated formatting and generated files are not included.

☐ Permissions — Admin and Sales permissions are correct in both navigation and protected backend endpoints.

☐ Validation — Frontend and backend validation rules are consistent; server-side validation is present.

☐ States — Loading, error, empty and success states are handled appropriately.

☐ Responsive UI — The relevant screen has been checked at suitable desktop and smaller viewport widths.

☐ Database/API — API or database changes are documented; migrations/setup notes are included where applicable.

☐ SQL safety — Parameterized queries are used; user-controlled values are not concatenated into SQL.

☐ Transactions — Stock/order changes use transactions where multiple database operations must succeed together.

☐ Frontend build — npm run build passes in frontend.

☐ Backend syntax — The relevant backend JavaScript passes node --check.

☐ Secrets and artifacts — No .env, credentials, node_modules, build output or mysql-data runtime files are included.

☐ Diff review — Changed files and the final diff have been reviewed for accidental changes.

☐ Evidence — The PR description explains how the change was tested and notes limitations or follow-up work.

> Review-ready means
> The reviewer can reproduce the change, understand its impact, see evidence of testing, and identify any remaining limitations without needing to guess.

Appendix A. Pull request description template

Copy this template into the pull request and replace the prompts with details specific to the change.

```
## Summary
- What changed?
- Why was the change needed?

## Implementation
- Frontend:
- Backend/API:
- Database/schema:

## Permissions and validation
- Admin behavior:
- Sales behavior:
- Validation and error handling:

## Testing
- [ ] Frontend build: `npm run build` (run in `frontend`)
- [ ] Backend syntax: `node --check <entry-file>`
- [ ] Manual happy-path test
- [ ] Invalid-input / boundary test
- [ ] Role and authorization test
- [ ] Responsive UI check (if UI changed)

## Documentation
- What docs or setup instructions changed?

## Risks / limitations
- Known limitations:
- Follow-up work:
```

Appendix B. Common review findings

Frontend-only protection

A restricted page is hidden, but its API endpoint accepts unauthorized requests.

Mismatched validation

The UI rejects a value that the backend accepts, or the UI accepts data the backend rejects without a clear explanation.

SQL interpolation

A request value is inserted directly into SQL instead of being passed as a query parameter.

Partial stock update

Order status changes even though one or more stock deductions fail.

Duplicate stock deduction

Repeating a completion request consumes stock more than once.

Missing empty/error state

The screen appears broken when no rows exist or an API call fails.

Undocumented schema change

A new column or table exists locally but setup instructions do not reproduce it.

Repository noise

The PR contains build output, local database files, credentials or unrelated formatting.

> Contribution mindset
> A good contribution is not just code that works once. It is a change that remains understandable, secure, testable and consistent with the rest of the system.